

$(-1)^0 = 1$ . If `upright` is `True`, then it will subtract  $n$  instead of adding  $n$ , which will enable us to go in the other direction. Let's check whether it can determine the entry above and to the right of 1 in the Luo Shu square:

---

```
print(rule1(1,3,True))
```

---

It should print 7, the correct value in the Luo Shu square.

For Rule 2, we can create an analogous function. Our Rule 2 function will take  $x$  and  $n$  as arguments, just like Rule 1. But Rule 2 is by default finding the entry below and to the right of  $x$ . So we will add an `upleft` argument that will be `True` if we want to reverse the rule. The final rule is as follows:

---

```
def rule2(x,n,upleft):  
    return((x + ((-1)**upleft))%n**2)
```

---

You can test this on the Luo Shu square, though there are only two pairs of entries for which this doesn't run into the exception to Rule 2. For this exception, we can write the following function:

---

```
def rule3(x,n,upleft):  
    return((x + ((-1)**upleft * (-n + 1)))%n**2)
```

---

This rule needs to be followed only when we're crossing the magic square's antidiagonal. We'll see later how to determine whether or not we are crossing the antidiagonal.

Now that we know how to fill the five central squares, and we have a rule to fill out the remaining squares based on knowledge of those central squares, we can fill out the rest of the square.

## Filling in the Rest of the Square

One way to fill in the rest of the square is to "walk" randomly through it, using known entries to fill in unknown entries. First, we'll determine the indices of our central entry as follows:

---

```
center_i = math.floor(n/2)  
center_j = math.floor(n/2)
```

---

Then, we can randomly select a direction to "walk," as follows: